

School ERP — Complete Technical Documentation

Version: 1.0.0-beta

Last Updated: 2024

Author: Generated for Faizabad Public School ERP

Classification: Technical Reference — Developer Handbook

Table of Contents

- [1. Project Overview](#)
 - [2. Technology Stack](#)
 - [3. Current Development Status](#)
 - [4. Completed Phases](#)
 - [5. Folder Structure](#)
 - [6. Database Schema](#)
 - [7. User Roles and Permissions](#)
 - [8. Authentication Flow](#)
 - [9. Student Management Module](#)
 - [10. Attendance Module](#)
 - [11. Examination Module](#)
 - [12. Fee Management Module](#)
 - [13. Homework & Notices Module](#)
 - [14. Timetable Module](#)
 - [15. API Endpoints Reference](#)
 - [16. Environment Variables](#)
 - [17. Deployment Architecture](#)
 - [18. Future Roadmap](#)
 - [19. Known Limitations](#)
 - [20. Multi-School Migration Plan](#)
 - [21. Installation Instructions](#)
-

1. Project Overview

What Is This System?

This is a **full-stack School ERP (Enterprise Resource Planning)** web application built to manage the complete operations of a school — from student admissions and daily attendance to examination results, fee collection, homework, and notices.

Business Purpose

The system is designed as a **commercial SaaS product** that can be sold to schools. The current build supports a single school (Faizabad Public School), but the architecture is deliberately built to support **multi-tenancy** (multiple schools) with minimal code changes.

Who Uses It?

Role	Description
Admin	School administration — full access to everything
Teacher	Class teachers and subject teachers — limited to their classes
Student	Students — view-only access to their own data
Parent	Parents — view-only access to their child's data

Design Philosophy

- **Modern over legacy** — Design quality comparable to Notion, Linear, Stripe dashboards
 - **Mobile-first** — Every screen works on a phone without a separate mobile app
 - **Data isolation** — Teachers only see their classes. Students only see their own data
 - **Audit trail** — Every write action is logged with who did it and when
 - **Future-ready** — Built to add new schools, modules, and features without rebuilding
-

2. Technology Stack

Frontend

Technology	Version	Purpose
Next.js	15.x	Full-stack React framework with App Router
React	19.x	UI component library
TypeScript	5.x	Type safety across entire codebase
Tailwind CSS	3.x	Utility-first CSS styling
ShadCN UI	Latest	Pre-built accessible UI components
Recharts	2.x	Charts and data visualization
React Hook Form	7.x	Performant form management

Zod	3.x	Runtime validation + TypeScript types
TanStack Table	8.x	Sortable, filterable, paginated data tables
date-fns	4.x	Date formatting and manipulation
Lucide React	Latest	Icon library

Backend

Technology	Version	Purpose
Next.js API Routes	15.x	RESTful API under <code>/api/v1/</code>
Prisma ORM	5.x	Type-safe database client + migrations
PostgreSQL	15.x	Primary relational database
Iron Session	8.x	Secure server-side sessions (httpOnly cookies)
bcryptjs	2.x	Password hashing (12 rounds)

Infrastructure

Service	Purpose
Vercel	Hosting, CI/CD, edge functions
Supabase	Managed PostgreSQL + file storage
GitHub	Version control, code repository

Key Libraries

Library	Purpose
<code>next-themes</code>	Dark/Light mode switching
<code>zustand</code>	Lightweight client state management
<code>geist</code>	Typography font (Vercel's font)
<code>tailwindcss-animate</code>	Animation utilities

3. Current Development Status

Overall Progress:  ~55% Complete

Phase	Module	Status	Completion
Phase 1	Foundation + Auth System	✓ Complete	100%
Phase 2	Student & Teacher Management	✓ Complete	100%
Phase 3	Attendance System	✓ Complete	100%
Phase 4	Examination System	✓ Complete	100%
Phase 5	Fee Management	↻ In Progress	0%
Phase 6	Homework & Notices	● Pending	0%
Phase 7	Timetable Management	● Pending	0%
Phase 8	Reports & PDF Export	● Pending	0%
Phase 9	Settings & Admin Tools	● Pending	0%
Phase 10	Analytics & Charts	● Pending	0%

4. Completed Phases

Phase 1 — Foundation & Authentication

What was built:

- Complete database schema with 16+ models
- Role-based session management using Iron Session
- Login page with mobile number as User ID
- First-login password change with strength meter
- Middleware protecting all routes by role
- 4 role dashboards (Admin, Teacher, Student, Parent)
- Sidebar navigation (collapsible on desktop, drawer on mobile)
- Dark/light mode toggle
- Audit logging system
- Database seeder with 33 real teachers, sample students, classes
- All auth API routes: [/api/auth/login](#), [/logout](#), [/change-password](#), [/me](#)

Files:

src/middleware.ts

src/lib/prisma.ts, session.ts, password.ts, utils.ts, validations.ts
src/app/(auth)/login/page.tsx
src/app/(auth)/change-password/page.tsx
src/app/api/auth/login|logout|change-password|me/route.ts
src/components/layout/Sidebar.tsx, TopBar.tsx
src/components/dashboard/KPICard.tsx, AttendanceChart.tsx, FeeChart.tsx
src/components/providers/ThemeProvider.tsx

Phase 2 — Student & Teacher Management

What was built:

- Student list with search, filter by class/gender, pagination
- Add student form that simultaneously creates Student + Parent accounts
- Credentials displayed once after creation (never shown again)
- Student profile page with attendance stats, fee summary, exam results
- Teacher list as card grid showing assigned subjects and classes
- Reset password (generates new 4-digit password, shown once to admin)
- Deactivate/activate student or teacher (blocks login)
- Reusable `DataTable` component with TanStack Table
- Reusable `PageHeader`, `StatusBadge` components
- Module structure: `src/modules/students/`, `src/modules/teachers/`
- API routes: `/api/v1/students`, `/api/v1/students/[id]`

Phase 3 — Attendance System

What was built:

- Interactive roll-call grid with P/A/L/E buttons per student
- "All Present" bulk button with individual override
- Upsert logic — safe to re-save, updates existing records
- Daily overview: all classes shown as cards with present/absent counts
- Class selector with date navigation (prev/next day, skip Sunday)
- Monthly attendance matrix report (students × days, color-coded)
- Student personal attendance calendar (month view)
- 75% attendance warning alert for students
- Teacher panel: sees only their assigned classes
- API routes: `/api/v1/attendance` (GET/POST)
- Module: `src/modules/attendance/`

Phase 4 — Examination System

What was built:

- Create exam with name, type, start/end dates
- Add multiple subjects with individual max marks and passing marks

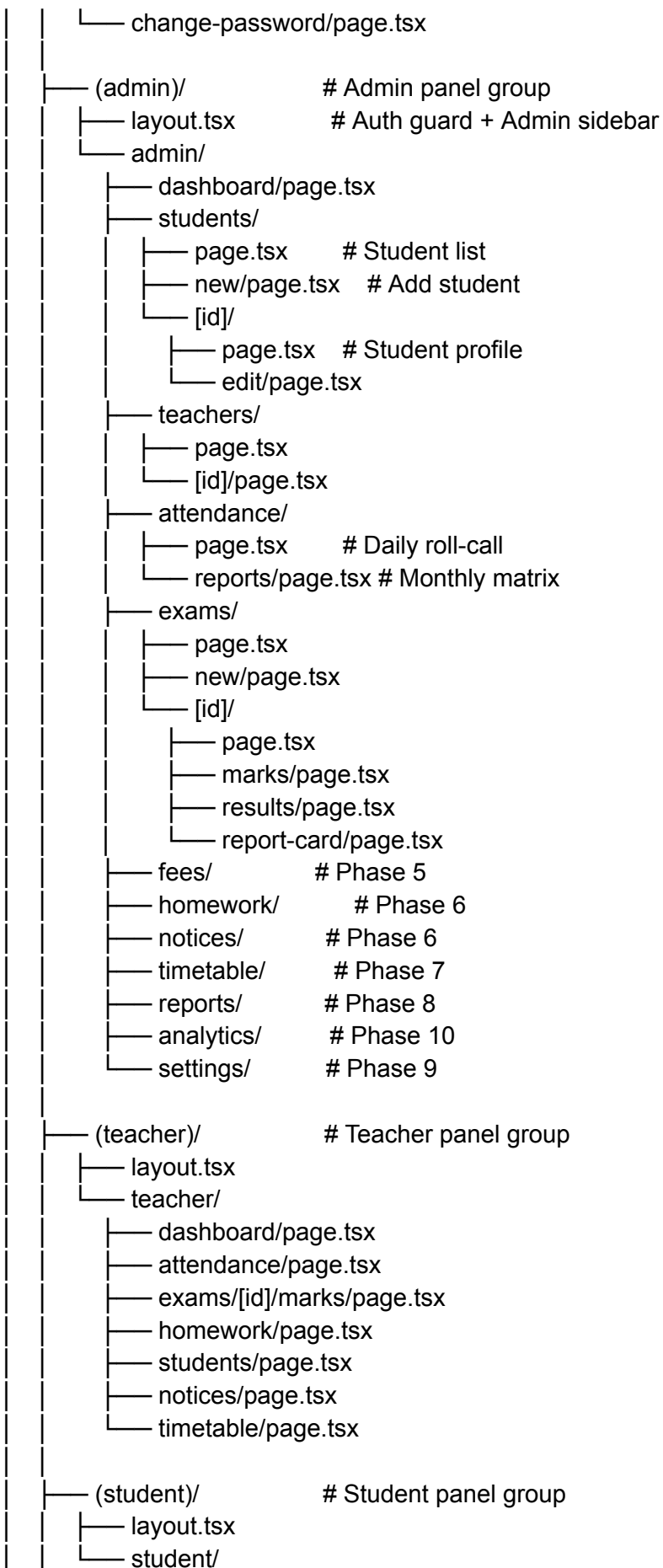
- Marks entry table with keyboard navigation (Enter/Tab moves to next student)
 - Live grade preview as marks are typed
 - Absent checkbox — marks null, shows AB in results
 - Validation — cannot enter marks exceeding max
 - Automatic grade calculation (A+/A/B+/B/C/D/F) from percentage
 - Rank generation sorted by total marks
 - Top 3 podium display (🏆 🥈 🥉)
 - Full results matrix table (students × subjects)
 - Pass/Fail per subject and overall
 - Report card generation (data layer complete)
 - Student results view: all exams grouped, subject breakdown, progress bar
 - API routes: `/api/v1/exams`, `/api/v1/exams/[id]`,
`/api/v1/exams/[id]/marks`
 - Module: `src/modules/exams/`
-

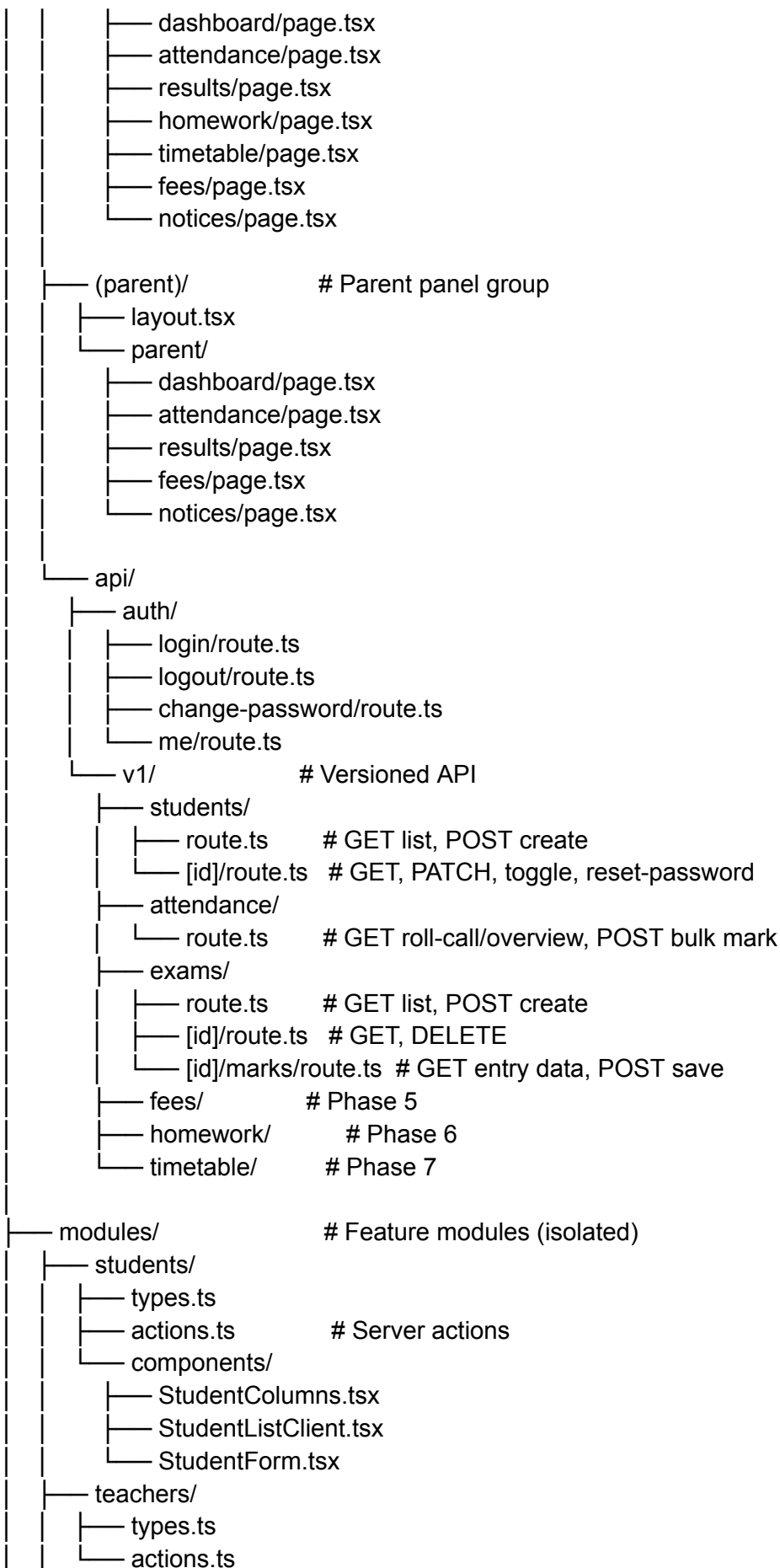
5. Folder Structure

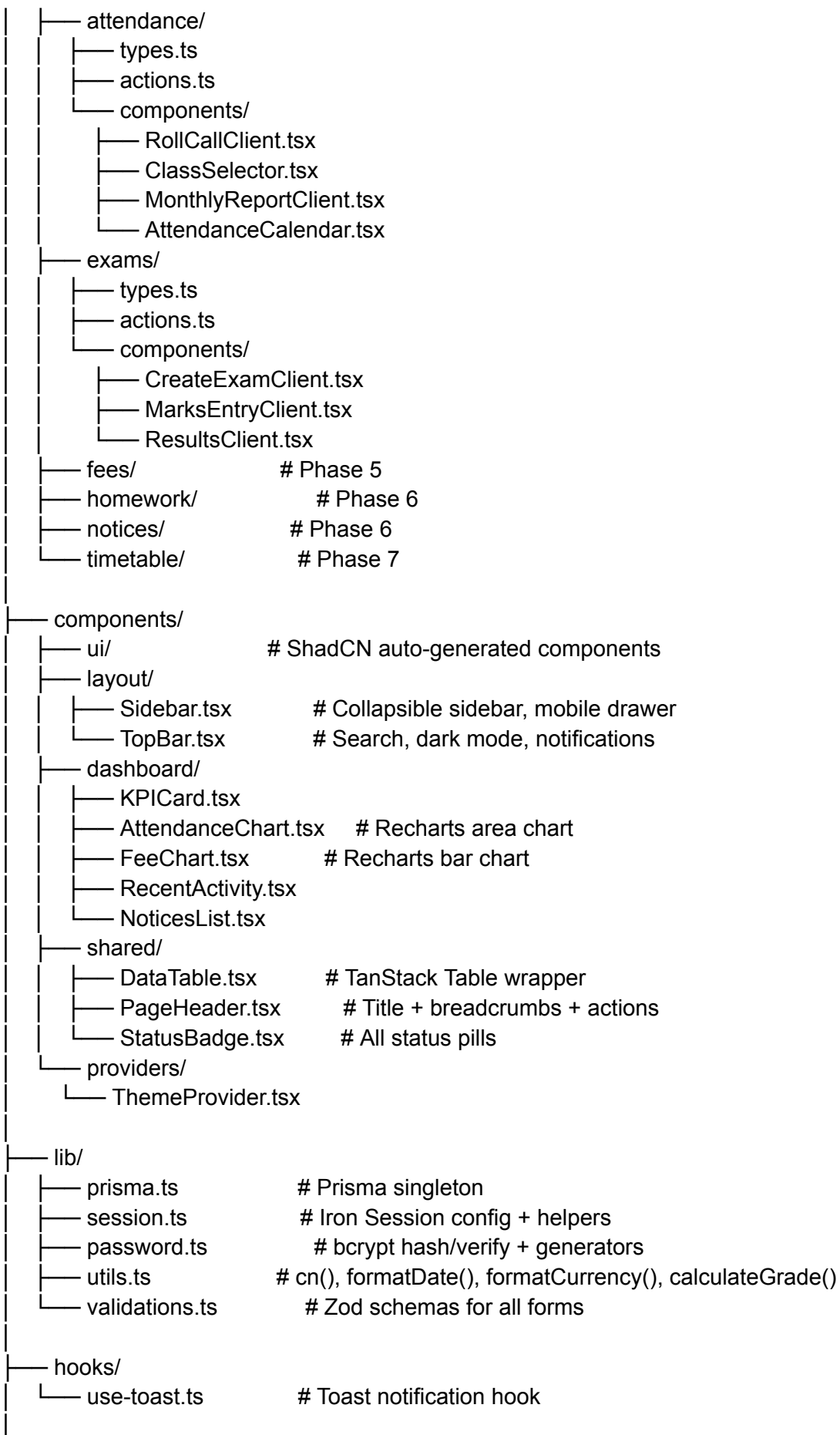
```

school-erp/
├── .env.local                # Local environment (never commit)
├── .env.example             # Template for new developers
├── .gitignore
├── next.config.ts
├── tailwind.config.ts
├── tsconfig.json
├── package.json
├──
├── prisma/
│   ├── schema.prisma        # Complete database schema
│   ├── seed.ts              # Sample data (33 teachers, students, classes)
│   └── migrations/          # Migration history (use: prisma migrate dev)
├── public/
│   ├── logo.svg
│   └── favicon.ico
├── src/
│   ├── middleware.ts         # Route protection (role-based)
│   ├──
│   ├── app/                 # Next.js 15 App Router
│   │   ├── globals.css      # CSS variables, dark mode, animations
│   │   ├── layout.tsx       # Root layout + ThemeProvider
│   │   └── page.tsx         # Root redirect → role dashboard
│   ├──
│   ├── (auth)/              # Auth group — no sidebar
│   │   └── login/page.tsx

```







└─ types/ # Global TypeScript types
└─ (module types live inside each module)

6. Database Schema

Core Models Summary

User ————— Authentication (mobile = User ID)

└─ Admin School administrator profile
└─ Teacher Teacher profile + qualifications
└─ Student Student profile + academic records
└─ Parent Parent profile linked to children

School School configuration + settings

└─ AcademicSession Academic year (2024-25 etc.)
└─ Class Class + Section (e.g. Class 10 A)

Subject Subject master list

SubjectAssignment Teacher ↔ Subject ↔ Class mapping

Attendance Daily per-student attendance record

Exam Examination (Unit Test, Half Yearly etc.)

└─ ExamSubject Subject in exam with max/passing marks
└─ ExamResult Per-student marks + grade

FeeStructure Fee template per class

└─ FeePayment Actual payment record per student

Homework Homework assigned by teacher

└─ HomeworkSubmission Student submission tracking

Notice Announcements with audience targeting

Timetable Class weekly schedule

└─ TimetableSlot Individual period (subject + teacher)

Document Student document uploads

AuditLog System-wide action log

Session Active login sessions

Key Relationships

User (1) ————— (1) Teacher

User (1) ————— (1) Student

User (1) ————— (1) Parent

User (1) ————— (1) Admin

Student (many) — (1) Class
 Student (many) — (1) Parent
 Student (1) — (many) Attendance
 Student (1) — (many) ExamResult
 Student (1) — (many) FeePayment

 Teacher (1) — (many) SubjectAssignment
 Teacher (1) — (1) Class [classTeacher]

 Class (1) — (many) Student
 Class (1) — (1) AcademicSession
 Class (1) — (many) SubjectAssignment

 Exam (1) — (many) ExamSubject
 ExamSubject (1) — (many) ExamResult

Grade Calculation Logic

Percentage	Grade	Points
90 – 100	A+	10
75 – 89	A	9
60 – 74	B+	8
50 – 59	B	7
40 – 49	C	6
33 – 39	D	5
Below 33	F	0 (FAIL)
Absent	AB	—

7. User Roles and Permissions

Permission Matrix

Feature	Admin	Teacher	Student	Parent
View all students	✓	✗	✗	✗
View class students	✓	✓ own	✗	✗
Add/Edit/Delete students	✓	✗	✗	✗
View all teachers	✓	✗	✗	✗

Add/Edit/Delete teachers	✓	✗	✗	✗
Mark attendance	✓	✓ own	✗	✗
View attendance (all)	✓	✓ class	✗	✗
View attendance (own)	—	—	✓	✓ child
Create exams	✓	✗	✗	✗
Enter marks	✓	✓ own sub	✗	✗
View results (all)	✓	✓ class	✗	✗
View results (own)	—	—	✓	✓ child
Manage fees	✓	✗	✗	✗
View fees (own)	—	—	✓	✓ child
Create homework	✓	✓	✗	✗
View homework	✓	✓	✓ own	✓ child
Create notices	✓	✓	✗	✗
View notices	✓	✓	✓	✓
Reset passwords	✓	✗	✗	✗
School settings	✓	✗	✗	✗
Analytics/Reports	✓	✓ class	✗	✗

Route Protection

/admin/* → requires role=ADMIN (middleware enforced)
 /teacher/* → requires role=TEACHER (middleware enforced)
 /student/* → requires role=STUDENT (middleware enforced)
 /parent/* → requires role=PARENT (middleware enforced)
 /api/v1/* → validates session cookie + role

8. Authentication Flow

Login Flow

1. User visits app → middleware checks session cookie
2. No session → redirect to /login
3. User enters mobile (10 digits) + password
4. POST /api/auth/login
5. Server: find user by mobile → verify bcrypt hash
6. If invalid → return 401 error
7. If inactive → return 403 error
8. If valid → create Iron Session (httpOnly cookie, 7-day expiry)
9. Session stores: userId, mobile, role, name, isFirstLogin
10. If isFirstLogin=true → redirect to /change-password
11. Else → redirect to /{role}/dashboard

First Login Flow

Admin creates Teacher/Student:

1. System generates 4-digit password (1000–9999)
2. Password hashed with bcrypt (12 rounds)
3. Hash stored in database
4. Plain password shown ONCE to admin → never stored again
5. isFirstLogin = true set on User record

Teacher/Student first login:

1. Enter mobile + auto-generated password
2. Successful auth → isFirstLogin=true detected
3. Middleware forces redirect to /change-password
4. User sets new password (min 6 chars) with strength meter
5. Server: verify current password → hash new password
6. isFirstLogin set to false
7. Redirect to role dashboard

Session Structure

```
interface SessionData {  
  userId:   string // User.id (cuid)  
  mobile:   string // 10-digit mobile (login ID)  
  role:     Role   // ADMIN | TEACHER | STUDENT | PARENT  
  name:     string // Display name  
  isFirstLogin: boolean // Forces password change  
  isLoggedIn: boolean // Session validity flag  
}
```

Password Security

Algorithm: bcrypt

Salt rounds: 12

Auto-generated: 4 digits (1000–9999)

Minimum user password: 6 characters

Storage: Hash only — plain text never persisted

Reset: Admin generates new 4-digit password → shown once

9. Student Management Module

Location

src/modules/students/

src/app/(admin)/admin/students/

src/app/api/v1/students/

Features

- **Student List** — paginated table (20/page), sortable, searchable by name/mobile/admission no
- **Filters** — by class, gender, active status
- **Add Student** — atomic transaction creates Student + Parent + 2 User accounts simultaneously
- **Credentials** — shown once after creation (student mobile+password, parent mobile+password)
- **Student Profile** — personal info, contact, class, attendance summary, exam results, fee status
- **Edit Student** — update all fields except mobile (User ID is permanent)
- **Reset Password** — admin generates new 4-digit password, shown once
- **Deactivate/Activate** — disables login without deleting data

Admission Number Format

ADM-{YEAR}-{5-digit-sequence}

Example: ADM-2024-00001

Key Actions (Server Actions)

```
getStudents(filters)      // paginated list
getStudentById(id)       // full profile with relations
createStudent(payload)    // atomic: student + parent
updateStudent(id, payload) // partial update
toggleStudentActive(id, bool) // deactivate/reactivate
resetStudentPassword(id)  // new 4-digit password
```

10. Attendance Module

Location

src/modules/attendance/
src/app/(admin)/admin/attendance/
src/app/(teacher)/teacher/attendance/
src/app/(student)/student/attendance/
src/app/api/v1/attendance/

Features

- **Daily Roll-Call** — per-student P/A/L/E buttons with visual feedback
- **Bulk Mark** — "All Present" button marks entire class, exceptions overridden individually
- **Daily Overview** — all classes shown as cards with present/absent counts and progress bars
- **Class + Date Navigation** — dropdown + prev/next day buttons, Sunday auto-skipped
- **Monthly Report** — scrollable matrix (students as rows, dates as columns)
- **Color Coding** — P=green, A=red, L=amber, E=blue across all views
- **Student Calendar** — personal month view with color-coded days
- **75% Alert** — student sees warning if attendance drops below threshold
- **Upsert Logic** — re-saving attendance updates existing records (safe to call multiple times)

Attendance Status Values

PRESENT → P → Green
ABSENT → A → Red
LATE → L → Amber
EXCUSED → E → Blue

Teacher Scope

Teacher only sees classes assigned to them via SubjectAssignment
or the class they are Class Teacher of

11. Examination Module

Location

src/modules/exams/
src/app/(admin)/admin/exams/
src/app/(teacher)/teacher/exams/
src/app/(student)/student/results/
src/app/api/v1/exams/

Features

- **Create Exam** — name, type, date range, multiple subjects with max/passing marks
- **Marks Entry** — keyboard-navigable table (Enter/Tab = next student)
- **Live Grade Preview** — grade shown in real-time as marks are typed
- **Absent Handling** — checkbox marks student absent (null marks, AB grade)
- **Validation** — marks cannot exceed max marks for subject
- **Auto Grade** — calculated from $(\text{marks}/\text{maxMarks} \times 100)$ percentage
- **Rank Generation** — sorted by total marks, assigned automatically
- **Merit Podium** — visual 🏆 🥈 🥉 for top 3 students
- **Results Table** — student × subject matrix with scrolling
- **Pass/Fail** — per subject (based on passing marks) and overall
- **Report Card** — data model complete (PDF rendering = Phase 8)
- **Student Results** — all exams grouped, subject breakdown, progress bar

Exam Types (Configurable in Phase 9 Settings)

UNIT_TEST → Unit Test

HALF_YEARLY → Half Yearly

ANNUAL → Annual Examination

PRE_BOARD → Pre-Board

BOARD → Board Exam

INTERNAL → Internal Assessment

Grade Boundaries

// src/lib/utils.ts — calculateGrade()

// Change these to update grades across entire system

90+ → A+

75+ → A

60+ → B+

50+ → B

40+ → C

33+ → D

<33 → F

12. Fee Management Module

Status: Phase 5 (In Progress)

Planned Features

- Fee structure setup per class and session

- Fee collection with payment mode (Cash, UPI, Cheque, Online, DD)
- Due fees list with overdue highlighting
- Receipt generation with unique receipt number
- Payment history per student
- Student and parent fee status view

Fee Receipt Number Format

RCP-{YEAR}-{6-digit-sequence}

Example: RCP-2024-000001

Fee Payment Status Values

PAID → Full amount received

PARTIAL → Some amount received

UNPAID → No payment yet

WAIVED → Fee waived by admin

13. Homework & Notices Module

Status: Phase 6 (Pending)

Homework Features (Planned)

- Teacher creates homework per subject per class
- Due date, description, optional file attachment
- Student views upcoming homework by subject
- Submission tracking (Pending/Submitted/Graded/Missed)

Notice Board Features (Planned)

- Admin/Teacher creates notices
- Audience targeting: ALL / TEACHERS / STUDENTS / PARENTS / CLASS
- Pin important notices to top
- Schedule future notices
- Notice expiry date

14. Timetable Module

Status: Phase 7 (Pending)

Planned Features

Authentication Endpoints

Student Endpoints

Attendance Endpoints

Exam Endpoints

GET	/api/v1/exams	List all exams for session ?sessionId=
POST	/api/v1/exams	Create new exam with subjects
GET	/api/v1/exams/:id	Get exam with subjects
DELETE	/api/v1/exams/:id	Delete exam and all results
GET	/api/v1/exams/:id/marks	Get marks entry data ?examSubjectId=&classId=
POST	/api/v1/exams/marks	Save bulk marks (upsert)

Response Format (All Endpoints)

// Success

```
{ success: true, data: T, total?: number, page?: number, totalPages?: number }
```

// Error

```
{ success: false, error: string, code: string }
```

// Error codes

UNAUTHORIZED → 401: Not logged in

FORBIDDEN → 403: Wrong role

NOT_FOUND → 404: Resource not found

VALIDATION_ERROR → 400: Invalid input data

DUPLICATE_MOBILE → 409: Mobile number already exists

INTERNAL_ERROR → 500: Server error

16. Environment Variables

— Database (Supabase PostgreSQL) —————

Connection pooling URL (for app queries via Prisma)

```
DATABASE_URL="postgresql://postgres:[PASSWORD]@db.[REF].supabase.co:5432/postgres?pgbouncer=true"
```

Direct connection URL (for migrations)

```
DIRECT_URL="postgresql://postgres:[PASSWORD]@db.[REF].supabase.co:5432/postgres"
```

— Session Security —————

Generate: openssl rand -base64 32

Minimum 32 characters

```
SESSION_SECRET="your-256-bit-random-secret-key-here"
```

— Application —————

```
NEXT_PUBLIC_APP_URL="https://yourdomain.com"
```

```
NEXT_PUBLIC_APP_NAME="SchoolERP"
```

```
NEXT_PUBLIC_SCHOOL_NAME="Faizabad Public School"
```

— Supabase Storage (file uploads) —————

```
NEXT_PUBLIC_SUPABASE_URL="https://[REF].supabase.co"
```

```
NEXT_PUBLIC_SUPABASE_ANON_KEY="your-anon-key"
```

```
SUPABASE_SERVICE_ROLE_KEY="your-service-role-key"
```

— Future: MSG91 OTP (placeholder — not implemented yet) —

```
# MSG91_AUTH_KEY=""
```

```
# MSG91_TEMPLATE_ID=""
```

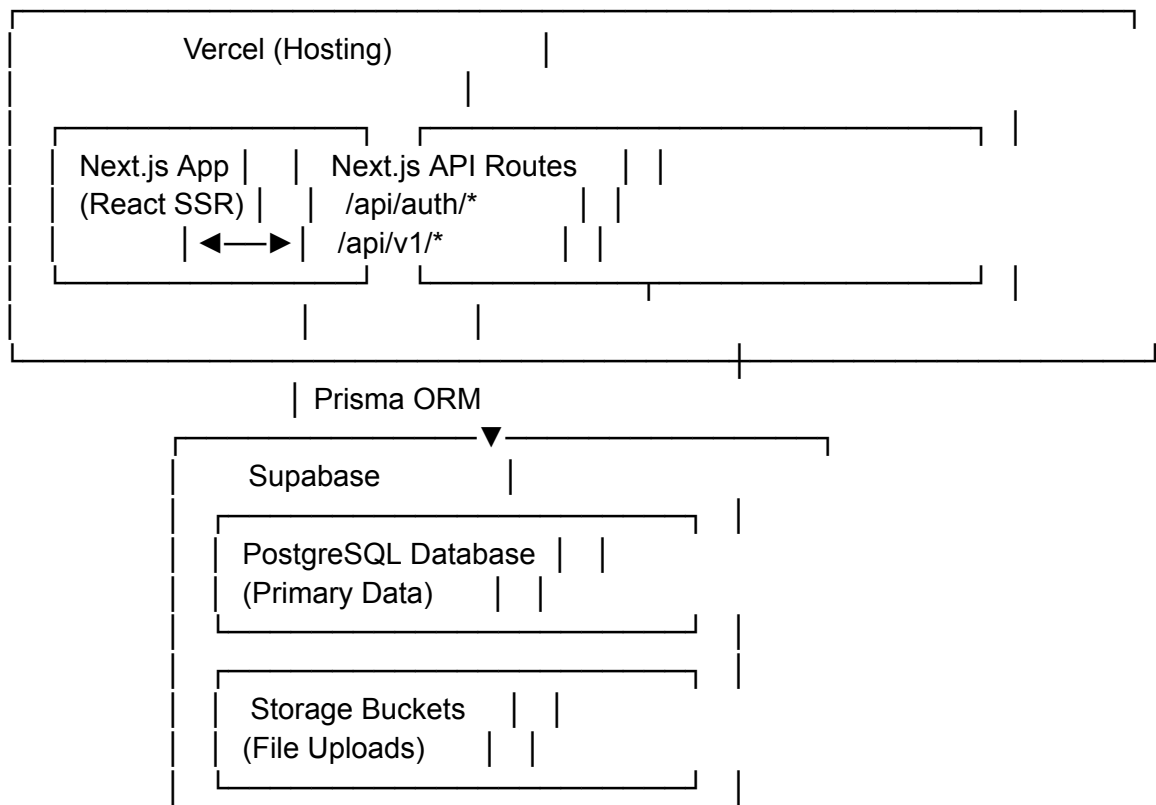
```
# MSG91_SENDER_ID=""
```

Variable Descriptions

Variable	Required	Purpose
DATABASE_URL	✔ Yes	Prisma database connection (pooled)
DIRECT_URL	✔ Yes	Direct DB for migrations
SESSION_SECRET	✔ Yes	Encrypts session cookies
NEXT_PUBLIC_APP_URL	✔ Yes	App base URL
NEXT_PUBLIC_SCHOOL_NAME	✔ Yes	Displayed in UI
NEXT_PUBLIC_SUPABASE_URL	For uploads	Supabase project URL
SUPABASE_SERVICE_ROLE_KEY	For uploads	Server-side file operations

17. Deployment Architecture

Current Architecture (Single School)

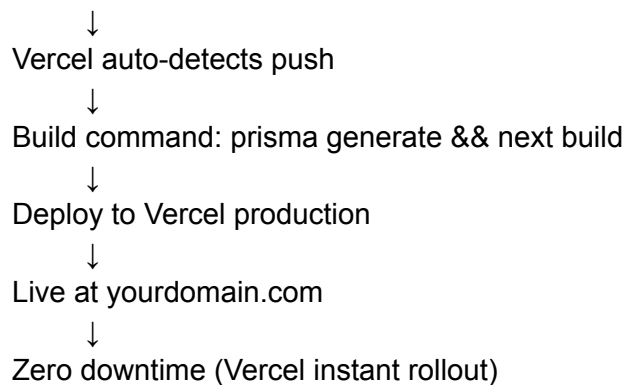


Request Lifecycle

1. Browser → HTTPS → Vercel Edge Network
2. Vercel → Next.js middleware (checks session cookie)
3. Middleware → allow/redirect based on role
4. Page renders (Server Component → Prisma query → HTML)
5. Client hydration → interactive components
6. API calls → /api/v1/* → Prisma → Supabase PostgreSQL

CI/CD Pipeline

Developer pushes to GitHub main branch



Cost at Scale

Users	Vercel Plan	Supabase Plan	Monthly Cost
1 school, <500 users	Free	Free	₹0
3-5 schools, <2000 users	Pro (\$20)	Pro (\$25)	~₹3,800
20+ schools, 10,000+ users	Pro (\$20)	Pro (\$25) + add-ons	~₹6,000-10,000

18. Future Roadmap

Phase 5 — Fee Management

- Fee structure per class and session
- Fee collection UI with receipt
- Due fee reports
- Payment history
- Student/parent fee view

Phase 6 — Homework & Notices

- Teacher creates and assigns homework
- Notice board with audience targeting
- Schedule and pin notices
- Student homework tracker

Phase 7 — Timetable Management

- Admin builds weekly timetable per class
- Teacher views their daily/weekly schedule
- Student views class timetable

Phase 8 — Reports & PDF Export

- Printable report cards (PDF via react-pdf)
- Attendance certificates
- Fee receipts (printable PDF)
- Excel export for attendance and marks
- Monthly attendance PDF reports

Phase 9 — Settings & Configuration

- School profile and logo
- Custom exam types (add/edit/remove Unit Test I, II, III, IV etc.)
- Academic session management
- Subject management (add/edit/remove)
- Class and section management
- Admin password reset for any user

Phase 10 — Analytics Dashboard

- Attendance trend charts (monthly)
- Fee collection vs dues graphs
- Student performance analysis
- Class comparison charts
- Subject-wise performance
- Year-over-year comparison

Phase 11 — PWA (Mobile App Feel)

- Progressive Web App setup
- Add to Home Screen prompt
- Offline attendance marking
- Push notifications (new homework, notices)
- App icon on phone home screen

Phase 12 — Multi-School (SaaS)

See Section 20 for complete migration plan.

Future Considerations

- SMS notifications via MSG91 (OTP placeholder already in code)
 - Parent-teacher chat
 - Bus tracking module
 - Library management
 - Transport fee management
 - Online fee payment (Razorpay integration)
 - React Native mobile app (after PWA)
-

19. Known Limitations

Limitation	Impact	Resolution Plan
Single school only	Cannot onboard multiple schools	Phase 12 multi-tenancy
No SMS/OTP yet	Forgot password not implemented	MSG91 slots ready in code
PDF generation pending	Report cards not printable yet	Phase 8 react-pdf
No file storage UI	Documents can't be uploaded via UI	Phase 8 Supabase Storage UI
Exam types hardcoded in form	Admin can't add custom types from UI	Phase 9 Settings
No real-time notifications	No live updates without page refresh	Phase 11 (WebSockets or polling)
No offline support	App requires internet connection	Phase 11 PWA service worker
Charts use sample data	Attendance/fee charts not from real DB	Phase 10 Analytics
Teacher marks entry scope	Teacher can currently enter marks for any class	To be restricted to assigned subjects only
No promotion module	Students not promoted to next class automatically	Phase 9 Settings

20. Multi-School Migration Plan

When To Do This

When you have 3+ schools and want to sell to more, implement multi-tenancy. **Estimated time: 3–5 days of development work.**

What Changes in the Database

```
// Step 1: Add Tenant model
model Tenant {
  id    String @id @default(cuid())
  name  String           // "Delhi Public School"
  slug  String @unique    // "delhi-public-school"
  domain String? @unique  // "delhi.yourapp.com"
  plan  Plan @default(BASIC)
  isActive Boolean @default(true)
  // ... billing, settings
}
```

```
// Step 2: Add tenantId to User
model User {
  tenantId String
  tenant Tenant @relation(...)
  // ... existing fields
}
```

```
// Step 3: Add tenantId to School
model School {
  tenantId String
  tenant Tenant @relation(...)
  // ... existing fields
}
```

What Changes in the Session

```
// Add tenantId to session
interface SessionData {
  userId: string
  tenantId: string // ← NEW
  role: Role
  name: string
  // ...
}
```

What Changes in API Queries

```
// BEFORE (current — single school)
const students = await prisma.student.findMany()
```



```
// AFTER (multi-tenant — all queries filtered)
const students = await prisma.student.findMany({
  where: { class: { session: { school: { tenantId: session.tenantId } } } }
})
```

Architecture For Multi-School

yourschoolerp.com → Landing + marketing page
app.yourschoolerp.com/login → Single login for all schools
(tenant detected from subdomain or email domain)

OR:

delhi.yourschoolerp.com → Delhi school login
ryan.yourschoolerp.com → Ryan school login
faizabad.yourschoolerp.com → FPS login

Pricing Model Suggestion

Basic Plan: Up to 300 students, 5 teachers, core modules → ₹2,000/month
Standard Plan: Up to 600 students, all modules → ₹4,000/month
Premium Plan: Unlimited students, custom branding, priority → ₹8,000/month

Super Admin Panel (Phase 12)

You (the developer/owner) get a Super Admin login:

- Create new school tenant
 - Manage subscriptions and plans
 - View usage across all schools
 - Suspend/activate schools
 - Access any school's data for support
-

21. Installation Instructions

Prerequisites

Node.js v20+
npm v10+
Git
A Supabase account (free)
A Vercel account (free)
A GitHub account (free)

Step 1 — Clone and Install

```
git clone https://github.com/yourusername/school-erp.git
cd school-erp
npm install
```

Step 2 — Install ShadCN Components

```
npx shadcn@latest init
# Style: Default | Color: Zinc | CSS Variables: Yes
```

```
npx shadcn@latest add button input label card badge avatar
npx shadcn@latest add dialog select separator dropdown-menu
npx shadcn@latest add toast tooltip tabs checkbox switch popover
```

Step 3 — Set Up Supabase

1. Go to supabase.com → New Project
2. Dashboard → Settings → Database
3. Copy "Connection string" → URI mode (with pgBouncer) → DATABASE_URL
4. Copy "Direct connection" → DIRECT_URL
5. Dashboard → Settings → API
6. Copy "Project URL" → NEXT_PUBLIC_SUPABASE_URL
7. Copy "anon public" key → NEXT_PUBLIC_SUPABASE_ANON_KEY
8. Copy "service_role secret" key → SUPABASE_SERVICE_ROLE_KEY

Step 4 — Environment Variables

```
cp .env.example .env.local
# Fill in all variables from Step 3
# Generate SESSION_SECRET:
openssl rand -base64 32
```

Step 5 — Database Setup

```
# Generate Prisma client
npx prisma generate

# Push schema to database (first time / development)
npx prisma db push

# OR run migrations (production)
npx prisma migrate deploy

# Seed with sample data (33 teachers, students, classes)
npm run db:seed
```

Step 6 — Run Development Server

```
npm run dev  
# Open http://localhost:3000
```

Step 7 — Test Login

Admin: 9999900000 / 1234
Teacher: (see console output after seed for all 33 teachers)
Student: (see console output after seed)

Step 8 — Deploy to Vercel

```
# 1. Push code to GitHub  
git add .  
git commit -m "feat: initial school erp deployment"  
git push origin main  
  
# 2. Go to vercel.com  
# 3. Import GitHub repository  
# 4. Add all environment variables from .env.local  
# 5. Set Build Command: prisma generate && next build  
# 6. Deploy  
  
# 7. Add custom domain  
# Settings → Domains → Add → yourschoolerp.com  
# Follow DNS instructions
```

Useful Commands

```
npm run dev          # Start development server  
npm run build        # Production build  
npm run db:seed      # Re-seed database  
npm run db:studio    # Open Prisma Studio (visual DB editor)  
npm run db:reset     # Reset DB + re-seed (DESTROYS DATA)  
npx prisma migrate dev --name "describe_change" # Create migration
```

Document End

This document should be updated whenever a new phase is completed.

For questions or issues, refer to the phase completion markdown files in the project root.

PHASE1_SETUP.md

PHASE2_COMPLETE.md

PHASE3_COMPLETE.md

PHASE4_COMPLETE.md

PROJECT_DOCUMENTATION.md ← this file